

memory allocation (B Tree)

```
template<typename D>
```

```
struct BTreeNode {
```

```
    unsigned short S;           // unsigned short S → 2 bytes    // padding → 6 bytes (for alignment)
```

```
    BTreeNode<D>* P;           // P pointer → 8 bytes
```

```
    D* KEY;                     // KEY pointer → 8 bytes
```

```
    BTreeNode<D>** LINK;        // LINK pointer → 8 bytes
```

```
                                // Total 32 bytes
```

```
    BTreeNode(int order, char* base) {                                // capacity 5
```

```
        KEY = (D*)( base + sizeof(BTreeNode<D>));
```

```
                                // 1000 → [ BTreeNode struct (32 bytes) ]
```

```
                                // block=1000, block+32=1032
```

```
                                // 1000 → [ BTreeNode struct (32 bytes) ]
```

```
                                // 1032 → [ start of KEY array ]
```

```
        LINK = (BTreeNode<D>**) ((char*)KEY + sizeof(D)*order);    // Address
```

```
    // default value
```

```
                                // 1000 → [ BTreeNode struct ]
```

```
    for(int i=0; i<order; i++) KEY[i] = D();
```

```
                                // 1032 → [ KEY array ]
```

```
    for(int i=0; i<=order; i++) LINK[i] = nullptr;
```

```
                                // 1052 → [ LINK array ]
```

```
    }
```

✓ Now KEY pointer points after the struct and LINK after KEY

```
};
```

```
template<typename D>
```

```
BTreeNode<D>* createNode(int order) {
```

```
    size_t size = sizeof(BTreeNode<D>) +
```

```
        sizeof(D) * order +
```

```
        sizeof(BTreeNode<D>*) * (order + 1);
```

```
                                // It computes how many bytes of memory you need for one node.
```

```
    char* block = new char[size];    // Type: char* → raw bytes → memory ready
```

```
    return new (block) BTreeNode<D>(order,block);    // Allocates one continuous block of memory
```

```
    // Size: exactly what we calculated
```

```
    // constructor → uses that memory (block)
```

```
}
```

```
BTreeNode<int>* node = createNode<int>(5);
```

```
delete[] (char*)node;
```

block

— [BTreeNode struct] ← this

S

P

KEY → points ↓

LINK → points ↓

— [KEY array (capacity elements)]

— [LINK array (capacity + 1 pointers)]

Address

1000 —————

[BTreeNode struct (32B)]

1032 —————

[KEY array (20B)]

1052 —————

[LINK array (48B)]

1100 ————— (end)

size =

sizeof(struct) // 32

+ sizeof(KEY array) // 20

+ sizeof(LINK array) // 48

= 100 bytes total

Version 1: Dynamic Arrays

```
template<typename D>
struct BTreeNode {
    unsigned short S;
    BTreeNode<D>* P;

    D* KEY;
    BTreeNode<D>** LINK;

    BTreeNode(int capacity) {
        KEY = new D[capacity];
        LINK = new BTreeNode<D>*[capacity + 1];
    }

    ~BTreeNode() {
        delete[] KEY;
        delete[] LINK;
    }
};
```

Analysis:

Memory:

- Worst memory efficiency
- 3 allocations: node, KEY, LINK
- Extra allocator overhead
- Fragmentation possible
- Scattered memory
- Memory waste = high

Performance:

- Slowest
- Multiple allocations
- Heap overhead
- Cache misses
- Pointer chasing

Version 2: Fixed Array

```
template<typename D, int MAX>
struct BTreeNode {
    unsigned short S;
    BTreeNode<D, MAX>* P;

    D KEY[MAX];
    BTreeNode<D, MAX>* LINK[MAX + 1];
};
```

Analysis:

Memory:

- Medium efficiency
- One allocation per node
- Internal unused space
- Wastes unused slots
- Memory waste = internal

Performance:

- Medium speed
- Single allocation
- Good cache locality
- Larger node size (fewer fit in cache)

Version 3: Single-Block Allocation

```
template<typename D>
struct BTreeNode {
    unsigned short S; BTreeNode<D>* P;

    D* KEY; BTreeNode<D>** LINK;
};

BTreeNode<int>* createNode(int capacity) {
    size_t size = sizeof(BTreeNode<int>) +
        sizeof(int) * capacity +
        sizeof(BTreeNode<int>*) * (capacity + 1);

    char* block = new char[size];

    BTreeNode<int>* node = (BTreeNode<int>*)block;

    node->KEY = (int*)(block + sizeof(BTreeNode<int>));
    node->LINK = (BTreeNode<int>**)(node->KEY + capacity);

    return node;
}
```

Analysis:

Memory:

- Best memory efficiency
- Single allocation
- Exact required size
- No fragmentation
- Minimal overhead
- Memory waste = minimal

Performance:

- Fastest
- One allocation
- Excellent cache locality
- Compact layout
- No pointer chasing